Blockchain-Based Library Book Lending Tracker

Technical Report, Final Summative Project

Software Engineering, Low-Level Specialization: Introduction to Blockchain Development

African Leadership University

Name: Golden Sash Munyankindi

Date: October 2026

GitHub repository: https://github.com/goldensash1/Final-Summative-Project-Blockchain-Based-Library-Book-Lending-Tracker

Demo video: [add the YouTube link of the demo video here]

Language and libraries: C11, compiled with gcc -Wall -Wextra (no warnings); OpenSSL 3 libcrypto for SHA-256 and ECDSA

Contents

1. Introduction
2. System Overview and Code Structure
3. Blockchain Architecture and Data Structures
4. Merkle Tree Construction and Verification
5. Copy Inventory Model and State Machine
6. Reservation Queue and Fines Lifecycle
7. Token Rewards, the Pending Pool and Mining Confirmation
8. UTXO Model
9. Account-Based Model
10. Comparison of the Two Transaction Models
11. Mining Simulation Methodology
12. ECDSA Signatures and Nonce Replay Protection
13. Misuse Detection and the Review Workflow
14. Chain Validation and Tamper Detection
15. Disk Persistence
16. Command-Line Interface
17. System Design Diagrams
18. Testing Results
19. Limitations and Conclusion

1. Introduction

Paper lending logs in a university library can be changed quietly. A librarian can mark a lost book as returned, a borrower can dispute what the log shows, or a due date can be moved after the fact. This project is a command-line program in C that keeps every lending event in a small blockchain, so a record cannot be changed after it is confirmed without the change being detected.

The summative project joins the work of the whole trimester into one system:

- **Formative 1:** book and member registries loaded from files, SHA-256 hashing, digitally signed lending records, chain validation and a CLI.
- **Formative 2:** a token reward for every return, a pending pool, proof-of-work mining (solo, pool and cloud), and two ways to keep token balances (UTXO and account-based).
- **Summative:** blocks with several records and a Merkle root, copy-level inventory, reservations, renewals, overdue detection and fines, a real ECDSA key pair for every member, nonce replay protection, a prioritised pending pool, fraud detection with a review step, full disk persistence, and a named-command CLI.

This report explains how each part works, why I made the main design choices, and how I tested the system.

2. System Overview and Code Structure

The program has 20 C source files (about 8,300 lines with headers). Each module has one job. The table lists them.

Table 1: Source modules

File(s)	What it does
main.c	Command-line options, the command table and the CLI loop, chain and tamper commands
app.c / app.h	Shared program state, startup, saving and loading, validate_chain, request submission, overdue scan
record.c	The lending record: signing payload, record hash, ECDSA sign and verify
merkle.c	Merkle tree construction and printing (written from scratch)
blockchain.c	Block, block hash, proof of work, chain checks
state.c	Copy inventory, availability state machine, reservation queues, nonces, fines
pending.c	Pending pool and its priority order
fraud.c	Fraud heuristics and the review commands
lending.c	Member, book, copy, lending, fine and history commands
mining.c	Building blocks, solo, pool and cloud mining, difficulty commands
tokens.c	Token commands (transfers, balances, history, replay test)
ledger.c, utxo.c, account.c, transaction.c	The confirmed transaction log and the two balance models
crypto.c, auth.c	SHA-256, ECDSA keys and keystore; librarian login with PBKDF2

registry.c, storage.c, input.c	Registry files, binary file storage, safe input and parsing

Librarians log in with a username and password (stored as a salted PBKDF2-HMAC-SHA256 hash). The logged-in librarian runs the commands, mines blocks and signs librarian actions. Members do not log in, but each member signs their own requests with their own private key (Section 12).

3. Blockchain Architecture and Data Structures

3.1 Block

A block has a header and a body. The body holds up to 8 lending records (3 by default). Table 2 shows the fields.

Table 2: Block fields

Field	Description
index	Position in the chain (0 = genesis)
timestamp	Unix time when the block was sealed. Must be strictly greater than the previous block's
record_count	Number of lending records in the block
previous_hash	SHA-256 hash of the previous block (64 zeros for genesis)
merkle_root	Root of the Merkle tree built from the hashes of the block's records
hash	SHA-256 of all header fields and the nonce
sealed_by	The miner or node that mined it (a username, POOL, or CLOUD:<username>)
token_reward	Coins for the block's returns: 10 per on-time return, 5 per late return, 0 if none
transaction_id	SHA-256 ID of the block's reward transaction (see below)
difficulty	Number of leading zeros this block was mined to
nonce	The value the miner changed until the hash met the difficulty
records[8]	The lending records (body)
reward_txids[8]	For each RETURNED record, the ID of the reward transaction the miner created

The block hash is SHA-256 of the text index|timestamp|record_count|previous_hash|merkle_root|sealed_by|token_reward|transaction_id|difficulty|nonce. The records are not hashed directly into the header. They are covered through merkle_root, so a change to any record changes the root.

A block can contain more than one return, but the brief gives the block one token_reward and one transaction_id. I handle this like this: token_reward is the sum of the rewards of the block's returns. If the block has one return, transaction_id is the SHA-256 ID of that reward transaction. If it has several, transaction_id is the SHA-256 of their IDs joined in block order. If it has none, it is 64 zeros. Validation recomputes both values from the records, so they cannot be changed on their own. The genesis block has no records. Its transaction_id points to the coinbase transaction that funds the LIBRARY reward treasury with 10,000 coins.

3.2 Lending record (transaction)

Table 3: Lending record fields

Field	Description
record_id	Unique request ID, e.g. REQ000012 (the request_id it was submitted with)
book_id, book_title	Must exist in the book registry; the title is copied from the registry
copy_id	The physical copy, e.g. BK001-C2 ('-' for a title reservation or a token transfer)
member_id, member_name	Must exist in the member registry; the name is copied from the registry
action	BORROWED, RETURNED, RESERVED, RENEWED or OVERDUE, plus LOST, FINE_PAID and TRANSFER
timestamp, due_date	When the request was made, and the loan's due date
fine_amount	0 unless the record is a late RETURNED, an OVERDUE, a LOST or a FINE_PAID record
condition, reason_code	Copy condition reported on return or loss; the librarian's override reason
token_reward	RETURNED only: 10 on time, 5 late
tx_id	Token transaction carried by a TRANSFER or token FINE_PAID record (64 zeros if none)
nonce	The signer's request counter (replay protection)
signer, signer_pubkey	Member ID, or LIB:<username> for librarian actions; the signer's public key
signature, sig_len	DER-encoded ECDSA P-256 signature over all the fields above

I added three actions to the five in the brief. LOST is the librarian override. FINE_PAID records a fine payment. TRANSFER is a member token transfer. The brief says all lending actions and token transactions must go through the pending pool, so they all need a record.

3.3 Pending request

A pending request holds the signed record plus pool data: status (PENDING, CONFIRMED or SUSPICIOUS), priority (overdue-related or normal), waiting_since (the reservation date or submission time), a submission sequence number, an approved flag, a duplicate flag, the fraud reason, and an optional token transaction. The brief's request_id, book_id, copy_id, member_id and action_requested are the record's own fields, so they are stored only once.

3.4 Registries and referential integrity

At startup the program reads books.txt (book_id,title,author), members.txt (member_id,full_name,course_code) and copies.txt (copy_id,book_id,condition). It prints an error and stops if a file is missing or empty. Malformed lines, duplicate IDs and copies of unknown books are skipped with a warning. IDs are stored in upper case. register_member, register_book and add_copy add new entries and append them to the same files, so they are loaded again on the next start.

Referential integrity is checked twice. When a request is made, the member, book and copy must exist. During validate_chain, every record on the chain must refer to a registered book and member.

3.5 Multiple records per block: fixed-size batching

I chose fixed-size batching. The miner takes the top N PENDING requests of the prioritised pool, where N is the batch size (default 3, set with --batch 1 to 8 and saved in meta.dat). mine_solo keeps building blocks until no eligible request is left, so a pool of 7 requests gives blocks of 3, 3 and 1. I chose a fixed size over a time window because it is deterministic. The same pool always gives the same blocks, which makes testing and the demo repeatable, and nothing depends on how long the program waits.

4. Merkle Tree Construction and Verification

merkle.c builds the tree from scratch. No external Merkle library is used.

1. Leaf: for every record, record_hash() takes SHA-256 of the full record text including its signature. The 32-byte digest is the leaf.
2. Parent: two neighbouring nodes are joined as raw bytes (32 + 32) and hashed: parent = SHA-256(left || right).
3. Odd count: if a level has an odd number of nodes, the last node is paired with itself, as in Bitcoin.
4. The levels are reduced until one node is left. That node is the Merkle root and is stored in the block header. A block with no records (the genesis block) has a root of 64 zeros.

The Merkle root is used in three places:

- **When a block is appended:** chain_append() recomputes the root from the records and refuses the block if it does not match merkle_root.
- **In validate_chain:** the root is recomputed for every block and compared with the stored value.
- **In tamper detection:** if someone edits a record but leaves the header alone, the block hash still matches because the header did not change. The recomputed Merkle root is different, so the change is caught. Section 14 shows this.

chain_view <index> prints each level of the tree, for example:

```
Merkle tree (leaf = SHA-256 of a signed record; parent = SHA-256(left||right)):
  Level 0 (leaves): a8a1b042b7ae d374c73fb086 20918b26a12d   <- REQ000001, REQ000002, REQ000003
  Level 1: 9b0b0ee89379 eca445e6c7de
  Level 2 (root): 7ffb5b4540c4
```

5. Copy Inventory Model and State Machine

Formative 1 tracked only whether a title was on loan. Now each physical copy is tracked. A copy has a copy_id, book_id, condition (NEW, GOOD, WORN or LOST), state (AVAILABLE, BORROWED, RESERVED or LOST), current_holder_id and due_date. It also stores the renewal count, an overdue flag, and the member it is held for when RESERVED.

Table 4: Valid state transitions

From	To	Trigger	Conditions
AVAILABLE	BORROWED	borrow_book	Copy available; due date after the borrow date
RESERVED	BORROWED	borrow_book	Only by the member the copy is held for
BORROWED	AVAILABLE	return_book	Returned by the holder; no reservation queue for the title
BORROWED	RESERVED	return_book	Queue not empty: offered to the first member in the queue
AVAILABLE / BORROWED	LOST	mark_lost	Librarian override with a reason code; holder fined 20 coins
BORROWED	BORROWED	renew_book / OVERDUE	Due date extended, or overdue flag set (state unchanged)

All transitions go through one function, state_apply() in state.c. It checks the full transition first and only then changes anything, so a rejected record leaves the state exactly as it was. Anything not in Table 4 is rejected: for example borrowing a BORROWED copy, returning a copy held by someone else, or marking a RESERVED copy as LOST.

The same function is used in three places:

- **At submission,** on a projected state: the confirmed state plus every pending request, in submission order. So two pending borrows of the same copy are impossible.
- **At mining,** on a copy of the confirmed state, to check every request that goes into the block.
- **In validate_chain,** to replay every record on the chain from the starting copies in copies.txt.

A copy added while the program is running is written to copies.txt with the index of the next block. During replay it only appears from that block on, so older records are still checked against the inventory as it was at that time.

Condition rules: a returned copy can stay the same or get worse (NEW, GOOD, WORN), but it cannot get better, and LOST can only be set with mark_lost. A drop of more than one level without a reason code is allowed but flagged as suspicious (Section 13).

6. Reservation Queue and Fines Lifecycle

6.1 Reservation queue

The queue is one array of (book_id, member_id, reservation_date) entries. The entries for one title keep their arrival order, so each title has its own first-in-first-out queue. reserve_book is accepted only when no copy of the title is AVAILABLE, the member is not already queued, and the member does not already hold or have a held copy of the title.

When a return is mined, the copy is offered automatically. The head of that title's queue is removed and the copy becomes RESERVED for that member, with the reservation date kept. Nobody else can borrow it. When the member borrows it, their request uses the reservation date

as its waiting time, so it is served in the order people waited. reservation_status shows the queue and the held copies.

6.2 Due dates and renewals

A loan is due 14 days after the borrow. A member may ask for a different due date with due=YYYY-MM-DD. The date must be a real calendar date (2026-02-30 is rejected), not in the past, and at most 60 days ahead. renew_book adds 7 days. It is refused if the copy was already renewed twice, if another member has reserved the title, or if the copy is already overdue.

6.3 Overdue detection and fines

Before every command, the program scans all BORROWED copies (overdue detection on each CLI invocation). For each copy past its due date that is not flagged yet, it creates an OVERDUE request signed by the librarian. The request records the fine so far (1 coin per day) and has overdue priority. So that this can be shown without waiting two weeks, advance_days <n> (ADMIN) moves a simulated library clock forward. The clock offset is saved in meta.dat. check_overdue lists every overdue copy with its days late and its fine so far.

Fine lifecycle:

1. While a copy is overdue, its fine grows by 1 coin per day (shown as Accruing in view_fines).
2. When it is returned late, the RETURNED record carries fine_amount = days late x 1 coin, and the reward is 5 instead of 10.
3. When that return is mined, the fine is added to the member's fines owed. A borrowed copy marked LOST adds a 20-coin fine.
4. settle_fine <member> [amount] [method=tokens|cash] creates a FINE_PAID request. With tokens it carries a transfer from the member to the LIBRARY account (plus the 1-coin fee). With cash only the record is kept. A payment larger than the amount owed is rejected.
5. When the FINE_PAID record is mined, the fines owed go down and the fines paid go up.

7. Token Rewards, the Pending Pool and Mining Confirmation

7.1 How returning a book creates a reward

1. return_book <copy_id> finds the copy's latest BORROWED record on the chain. If there is none (for example if the borrow is still pending), it prints an error.
2. It compares the time with the due date to decide if the return is on time or late, and sets token_reward (10 or 5) and fine_amount.

3. It builds a RETURNED record, signs it with the member's private key, checks it, and puts it in the pending pool. No balance changes yet.
4. When a miner puts the record in a block, the miner creates the reward transaction: LIBRARY pays the member the reward minus the 1-coin fee (9 or 4 coins). The fee goes to the miner. The transaction ID is SHA-256 of the transaction and is stored in reward_txids and in the block's transaction_id.
5. After the proof of work is found, the block is appended and the reward is confirmed in the ledger. Only then does the member's balance change. The miner is paid a coinbase of 50 coins per block plus the fees.

A book that stays overdue and is not returned creates no transaction. Its OVERDUE record has token_reward 0.

Design change from Formative 2: there, the reward transaction was created when the book was returned. Now the miner creates it when the return is confirmed. The pool is now prioritised, so returns can be mined in a different order from the one they were made in. In the account model, the LIBRARY account's transaction nonces must stay in order, and creating each reward at confirmation time keeps them in order. It also matches the brief, which rewards confirmed returns.

7.2 Pending pool management

- **Add:** every request passes the checks in Figure 2 (signature, nonce, valid transition, funds) before it enters the pool. The pool is saved to pending.dat after every change.
- **View:** pool_view lists every PENDING and SUSPICIOUS request in priority order, with its status and the fraud reason if it has one.
- **Prioritise:** the pool's indices are sorted with qsort: (1) overdue-related requests first (OVERDUE flags, returns of overdue copies, fine settlements), (2) waiting_since ascending, so the member who waited longest goes first, (3) submission order.
- **Select:** the miner takes the top N PENDING requests in that order. A request that cannot apply yet is deferred and the miner tries the next one. For example, a member's request with nonce 5 must wait until their request with nonce 4 is in a block. SUSPICIOUS requests are never selected.
- **Remove:** after the block is appended, its requests are marked CONFIRMED and removed from the pool.

8. UTXO Model

In the UTXO model (utxo.c) no balance is stored. The UTXO set is a list of unspent outputs (txid, vout, owner, amount). A member's balance is the sum of the outputs they own. Member IDs from the registry are the owners, and every member starts with 0.

- **Inputs and outputs:** a transaction spends whole earlier outputs as inputs, chosen oldest first, until they cover the amount plus the fee. Output 0 pays the recipient.
- **Fee:** every transaction has a fixed fee of 1 coin. The fee is not an output: it is the inputs minus the outputs, and the miner collects it.
- **Change:** if the inputs are worth more than amount + fee, output 1 returns the rest to the sender as a change UTXO.
- **Insufficient inputs:** the transaction is rejected.
- **Double spending:** while a transaction is pending, its inputs are locked (reserved_by), so no other pending transaction can use them. When it is confirmed they are removed from the set. replay_tx re-submits a confirmed transaction and it is rejected as a double spend.
- **Display:** the full UTXO set, with every output and its owner, is printed after each confirmed block.

Example from the demo run: ALU002 sends 3 coins to ALU005.

```
UTXO view of tx a6610df7:
  input  e39d80a1e89e:0  ALU002                9.00
  output [0]            ALU005               3.00
  output [1]            ALU002               5.00  (change back to sender)
  fee = inputs 9.00 - outputs 8.00 = 1.00 (collected by the miner)
```

9. Account-Based Model

In the account model (account.c) each account stores a balance, a nonce, the amount reserved by pending transactions, and a linked list of its transaction history (sender, recipient, amount, fee, nonce, time).

- **Nonce:** each account's nonce counts its outgoing transactions. A new transaction must use exactly the next nonce (confirmed + pending). A reused nonce (replay) or a skipped nonce is rejected. The nonce goes up when the transaction is confirmed.
- **Balance:** a transfer is rejected if the available balance (balance minus reserved) is lower than amount + fee. Balances are updated directly when a block is confirmed.
- **History:** every confirmed transaction is appended to the linked lists of both accounts. transaction_history <member_id> walks the list.
- **Commands:** account_transfer <from> <to> <amount> [nonce] lets the user type the nonce to test replay protection; account_balance and account_nonce show the table and one nonce.

Example from the demo run (account model):

```
> account_transfer ALU005 ALU006 1 3
REJECTED: token transaction: invalid nonce: 3 skips ahead of the next valid nonce 0 for ALU005
> account_transfer ALU005 ALU006 1 0
ACCEPTED: transfer 649e3b311856cd5d...: ALU005 -> ALU006, amount 1.00, fee 1.00, account nonce 0
> account_transfer ALU005 ALU006 50 1
REJECTED: token transaction: insufficient balance: ALU005 has 1.00 available ... needs 51.00
```

This account nonce protects token transactions. It is separate from the request nonce of Section 12, which protects every signed lending request.

10. Comparison of the Two Transaction Models

Table 5: UTXO and account models

Aspect	UTXO model	Account model
What is stored	Unspent outputs; balance = sum of a member's outputs	One balance per account
Spending	Whole outputs are spent; extra comes back as change	Balance is debited directly
Double spend / replay	A spent output no longer exists	The nonce must be exactly the next one
Pending transactions	Inputs locked; unconfirmed change can be chained	Amount reserved; pending count
History	Rebuilt by filtering the log	Linked list per account
Auditing	Every coin can be traced output to output	Easier to read: one number per member
Complexity	More complex (coin selection, change)	Simpler

The model is chosen for the whole session with --model utxo or --model account (or a menu at startup). Both models are rebuilt from the same list of confirmed transactions (ledger.dat). Starting the program with the other model therefore gives exactly the same balances, and the tests check this. For a library, the account model is easier for staff to understand, because each member has one balance. The UTXO model gives a stronger audit trail, because every coin can be followed back to the return that created it.

11. Mining Simulation Methodology

11.1 Proof of work and difficulty

To mine a block, the miner sets the block's position, difficulty and miner name, and then tries nonce = 0, 1, 2, ... until SHA-256 of the header starts with the required number of zero hex digits. Only the nonce changes between attempts, so the fixed part of the header is hashed once (a SHA-256 midstate) and each attempt only hashes the nonce. The number of attempts and the time are printed. The difficulty is 1 to 4, set with --difficulty N or set_difficulty N. Each block stores its own difficulty, so old blocks still validate after a change. Each extra zero makes a valid hash 16 times rarer. The benchmark command measured this:

Table 6: Measured mining effort

Difficulty	Target	Expected attempts	Average of 20	Min	Max	Avg time (ms)
1	0...	16	17	2	56	0.007
2	00...	256	344	18	1,026	0.102
3	000...	4,096	2,656	104	9,903	0.785
4	0000...	65,536	69,543	6,507	285,782	20.896

11.2 Solo mining

mine_solo: one miner (the logged-in librarian) builds and mines every block until no eligible request is left. It shows each block's records, nonce, attempts, time and hash, then the UTXO set or the updated accounts. The miner gets the full reward, 50 coins per block plus all fees, on top of the members' token rewards.

11.3 Pool mining

mine_pool [n] (2 to 8 miners, default 4): each miner gets a random hash rate of 100 to 500 attempts per round (--seed makes this repeatable). Miner k tries nonces k, k+n, k+2n, ..., so no work is repeated. In each round the miners' attempts are taken in time order, spread evenly over the round, and the first valid hash ends the search. So each miner's number of attempts follows its hash rate. At the end, a 2% pool fee is taken from the total reward, and every miner gets share = miner_attempts / total_attempts x (total reward - pool fee). Amounts are integer hundredths of a coin, and the rounding remainder goes to the pool operator. The result is printed as a table:

```
+-----------+-----------+------------+----------+--------+-------------+
| Miner ID  | Hash rate |   Attempts |  Share % | Blocks |      Reward |
| POOL_M1   |   234/rnd |         30 |   18.40% |      0 |        9.01 |
| POOL_M2   |   191/rnd |         25 |   15.34% |      1 |        7.51 |
| POOL_M3   |   481/rnd |         62 |   38.04% |      0 |       18.63 |
| POOL_M4   |   358/rnd |         46 |   28.22% |      0 |       13.82 |
| TOTAL     |           |        163 |  100.00% |      1 |       48.97 |
```

11.4 Cloud mining

mine_cloud <rounds> (1 to 5): the user rents 400 hash attempts per round for a fixed rental fee of 20 coins per round. Work on an unsolved block carries over to the next round. Each round, the mining rewards earned in that round are credited to the renter. The summary shows gross earnings, total fees paid and net profit for every round. If the cumulative fees are higher than the cumulative rewards after any round, a WARNING is printed. The rental fee is paid to the provider outside the chain, so the ledger receives the rewards.

Figure 5. Mining workflow: difficulty and the solo, pool and cloud paths

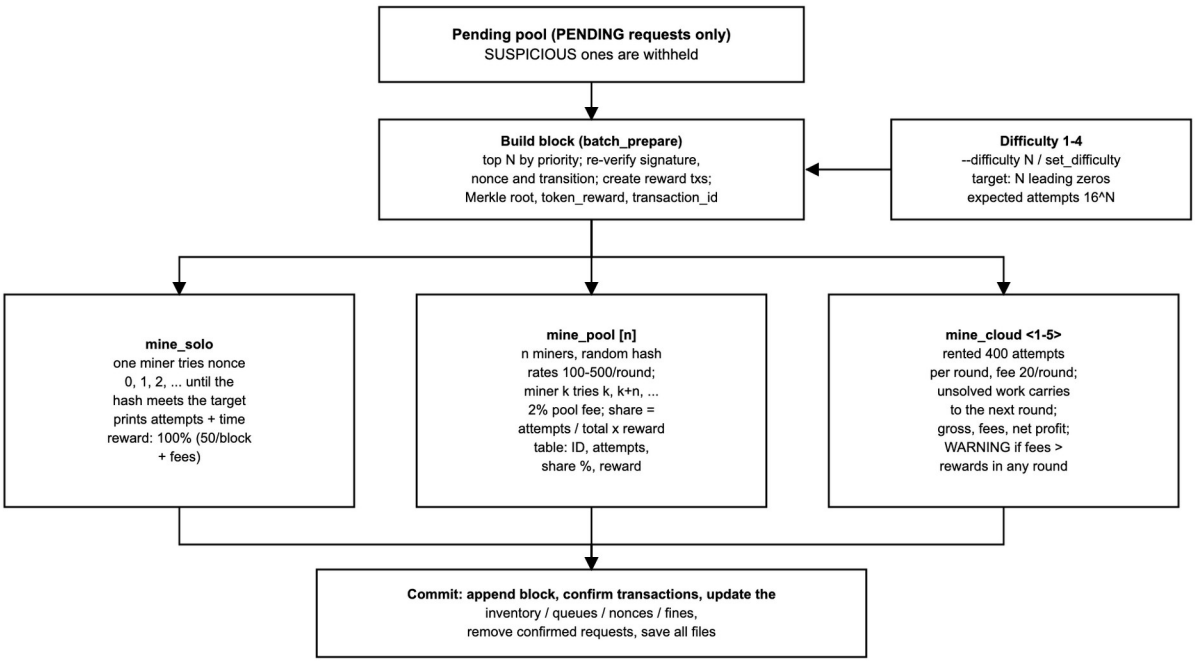


Figure 5. Mining workflow (also in Section 17)

12. ECDSA Signatures and Nonce Replay Protection

12.1 Key generation

All keys are ECDSA keys on the NIST P-256 curve (prime256v1), made with OpenSSL's EVP_PKEY_keygen.

- **Librarians:** a key pair is made when the account is created. The private key is saved AES-256 encrypted with the user's password (keys/<user>_private.pem, mode 0600) and unlocked at login.
- **Members:** every member in members.txt gets a key pair at startup if they do not have one, and register_member makes one for a new member (keys/members/<ID>_private.pem with mode 0600, and _public.pem).
- **On-chain identity:** the public key is exported as the 65-byte uncompressed point (04 || X || Y), written as 130 hex characters, and stored in every record as signer_pubkey.

The brief names the EC_KEY API. In OpenSSL 3 the EC_KEY functions are deprecated and give warnings under -Wall, so I used the EVP_PKEY interface, which is OpenSSL 3's replacement for the same ECDSA operations.

12.2 Signing

1. The record's fields (everything except the signature, but including the signer and their public key) are joined into one text payload.
2. The payload is hashed with SHA-256.
3. The hash is signed with the signer's private key (EVP_DigestSign with SHA-256). The DER signature (at most 72 bytes) is stored in the record.

Member requests (borrow, return, renew, reserve, fine payment, transfer) are signed with the member's own key. Librarian actions (the automatic OVERDUE flag and mark_lost) are signed with the logged-in librarian's key.

12.3 Verification

To verify, the public key is rebuilt from signer_pubkey (EVP_PKEY_fromdata) and the signature is checked against the hash of the payload. Then the embedded key is compared with the key registered for that signer in the key files. Without this second check, an attacker could sign a fake record with their own key and include that key. A record with a missing signature (length 0) or an invalid signature is rejected. Verification happens:

- before a request may enter the pending pool;
- again before the miner puts a request in a block (requests that fail are dropped);
- for every record of every block in validate_chain.

12.4 Nonce-based replay protection

- Every signer (member or librarian) has a request nonce that starts at 0. It is stored in the library state.
- Every request carries the signer's current nonce, and the nonce is covered by the signature.
- A request is rejected unless its nonce equals the signer's expected next nonce (confirmed nonce + the signer's pending requests). borrow_book ... nonce=N can be used to show this.
- The stored nonce only goes up when the record is included in a mined block.
- validate_chain replays every record and checks that each signer's nonces appear exactly once and in order, so a copied or re-ordered record on the chain is detected.
- If a pending request is rejected, the same signer's later requests are re-signed with the corrected nonces, so the sequence has no gap.

member_nonce <id> shows the confirmed nonce and the next nonce a request must use.

13. Misuse Detection and the Review Workflow

fraud.c checks every new request. If a check triggers, the request is still stored, but with status SUSPICIOUS and the reason, and it is never put in a block while it is SUSPICIOUS.

Table 7: Fraud heuristics

Heuristic	Rule in this program
Excessive concurrent borrows	The member already holds 3 copies (MAX_CONCURRENT_BORROWS) and borrows another
Rapid re-borrow pattern	The member borrows a title less than 24 hours after returning it
Duplicate request	The request_id already exists on the chain or in the pool (replay_request re-submits a captured record byte for byte)
Condition manipulation	A return lowers the copy's condition by more than one level (e.g. NEW to WORN) without a librarian reason code (reason=CODE)

Review workflow:

1. fraud_review lists every SUSPICIOUS request with its signer, nonce, signature status and reason.
2. approve_suspicious <id> sets it back to PENDING (marked as approved), and it is mined in priority order.
3. reject_suspicious <id> discards it. The pool is then checked again: the signer's later requests are re-signed with corrected nonces, and requests that are no longer valid are dropped.
4. A duplicate can only be rejected, never approved. Its request_id and nonce are already used.
5. mine_solo, mine_pool and mine_cloud print every withheld SUSPICIOUS request.

A SUSPICIOUS request also holds back the same member's later requests, because those have higher nonces and must wait for it. This is intended: a member under review cannot carry on until a librarian decides.

14. Chain Validation and Tamper Detection

validate_chain checks the whole chain:

- previous_hash of every block equals the hash of the block before (64 zeros for genesis);
- the recomputed block hash equals the stored hash;
- the hash meets the block's difficulty (proof of work);
- each block's timestamp is strictly greater than the previous block's;
- the Merkle root recomputed from the records equals merkle_root;
- every record's ECDSA signature is valid and made with the signer's registered key;
- token_reward and transaction_id match the block's RETURNED records;
- every record refers to a registered book and member;
- replaying every record through state_apply() succeeds (legal transitions, nonces in order);

- every transaction a record or block refers to is in the ledger with a matching SHA-256 ID and amount.

When a saved chain is loaded, validate_chain runs before anything else. If it fails, the session is locked: only viewing commands, validate_chain and chain_load work, and nothing is saved. tamper_block <index> <mode> changes the chain in memory to show this:

Table 8: Tamper demo modes

Mode	What it changes	What validate_chain reports
record	A record's due date; header untouched	Merkle root mismatch; record signature does not verify
header	The block's token_reward	Stored hash does not match recomputed hash
remine	A record; Merkle root recomputed; block re-mined	Signature fails; next block's previous_hash link broken

Changes made directly to the files are also caught. Editing one byte of a title in chain.dat gives a Merkle root mismatch and a bad signature at the next start. Replacing a member's public key file is reported as "signed by a key that is not the registered public key". chain_load reloads the saved files and re-validates.

15. Disk Persistence

I chose a binary format (.dat files). Every structure has a fixed size, so it is fast and exact to save, and there is no text parsing that could go wrong. storage.c writes every file in the same layout:

```
offset 0   4 bytes   magic, e.g. 'LBC3' for the chain
offset 4   uint32    record size in bytes (sizeof the struct)
offset 8   int32     record count
offset 12  ...       count x record size bytes of raw records
```

Each file is first written to <name>.tmp and then renamed, so a crash can never leave a half-written file. When a file is loaded, the magic, record size and count are checked, so a file from another version or a truncated file is refused with a message. All strings are then force-terminated, so a corrupted file cannot cause reads past the end of a field.

Table 9: Saved files

File	Magic	Contents	Record
chain.dat	LBC3	Every block with its records and reward IDs	Block, 5,600 bytes
pending.dat	LBP3	The pending pool (status, priority, fraud reason, tx)	PendingRequest, 3,456 bytes
ledger.dat	LBL2	Every confirmed token transaction, in order	Transaction, 2,664 bytes
library_state.dat	LBS1	Copy inventory, reservation queues, request nonces, fines	LibraryState (one record)
balances.dat	LBB1	Balance and account nonce of every account	id, balance, nonce
utxo.dat	LBU1	The confirmed UTXO set (UTXO sessions)	Utxo, 192 bytes
meta.dat	LBM1	Difficulty, block reward, batch size, clock offset, next request ID, height, tip hash	ChainMeta (one record)

Files are saved after every mined block, after every change to the pool, and on exit (chain_save also saves on request). At startup the program checks whether chain.dat exists. If it does not, it creates a new chain with a genesis block. If it does, it loads all the files, replays the ledger into the chosen model, runs validate_chain, and rebuilds the inventory, queues, nonces and fines from the chain. It then compares the rebuilt state with library_state.dat and balances.dat and prints a warning if they differ. The chain is the source of truth.

A limitation of the binary format is that the files are tied to the compiler's struct layout, so they cannot be moved to a machine with a different architecture. The magic and record-size check stops such a file from being read wrongly.

16. Command-Line Interface

Commands are typed one per line, with arguments, for example borrow_book ALU001 BK001. Missing arguments are asked for. help lists every command. Every command named in the brief exists, plus a few extras for testing and the demo (advance_days, tamper_block, replay_request, replay_tx, benchmark, list_books, list_members).

Table 10: Commands

Group	Commands
Membership	register_member, view_member, member_nonce, list_members
Books and copies	register_book, view_book, add_copy, view_copy, mark_lost, list_books
Lending	borrow_book, return_book, renew_book, reserve_book, reservation_status
Fines and overdue	check_overdue, view_fines, settle_fine, advance_days
Tokens and ledger	token_transfer, token_balance, utxo_view, account_balance, account_transfer, account_nonce, transaction_history, replay_tx
Chain and mining	pool_view, mine_solo, mine_pool, mine_cloud, chain_view, validate_chain, chain_save, chain_load, difficulty_status, set_difficulty, benchmark
Fraud and audit	fraud_review, approve_suspicious, reject_suspicious, replay_request, lending_history, member_history, book_history, tamper_block

Input validation: IDs must have the right format (3 to 15 letters and digits starting with a letter; copies <BOOK_ID>-C<n>) and must exist; dates must be real and not in the past; token amounts must be positive with at most two decimals; fines and token rewards can never be negative; all input lines have a maximum length.

17. System Design Diagrams

The six diagrams below show the design. The same files are in docs/diagrams/ in the repository.

Figure 1. Blockchain structure with the Merkle tree of each block

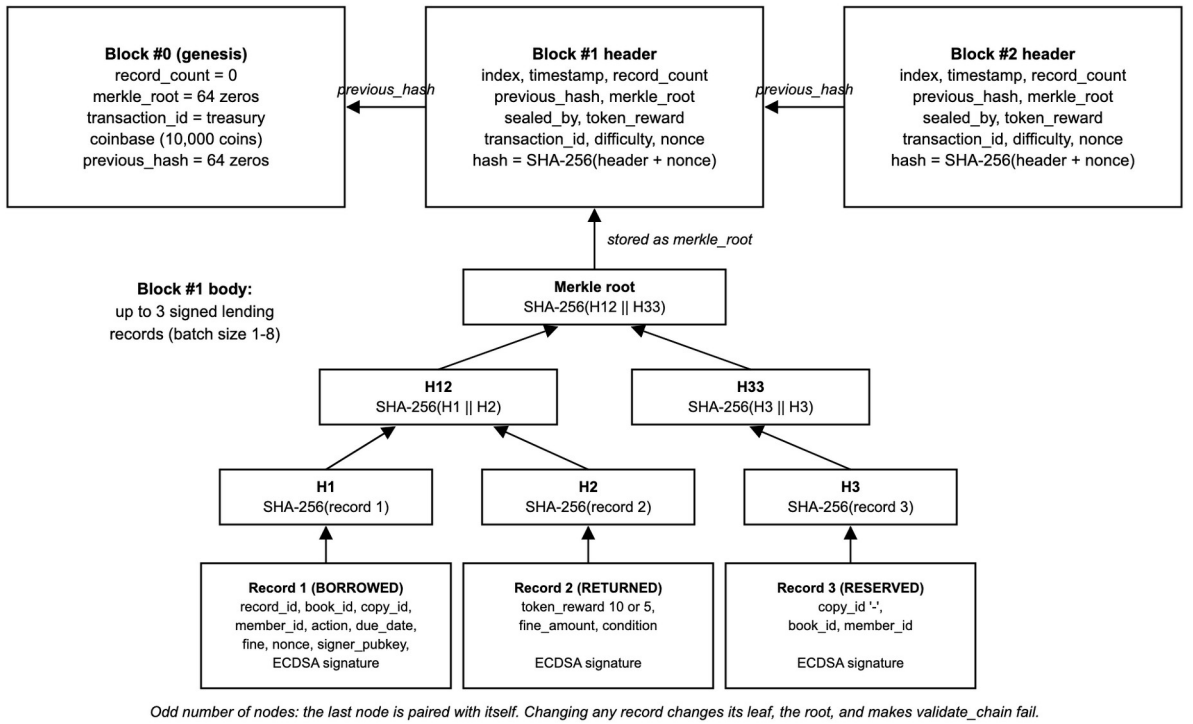


Figure 1. Blockchain structure with the Merkle tree

Figure 2. Pending pool flow: signing, checks, fraud flagging and prioritisation

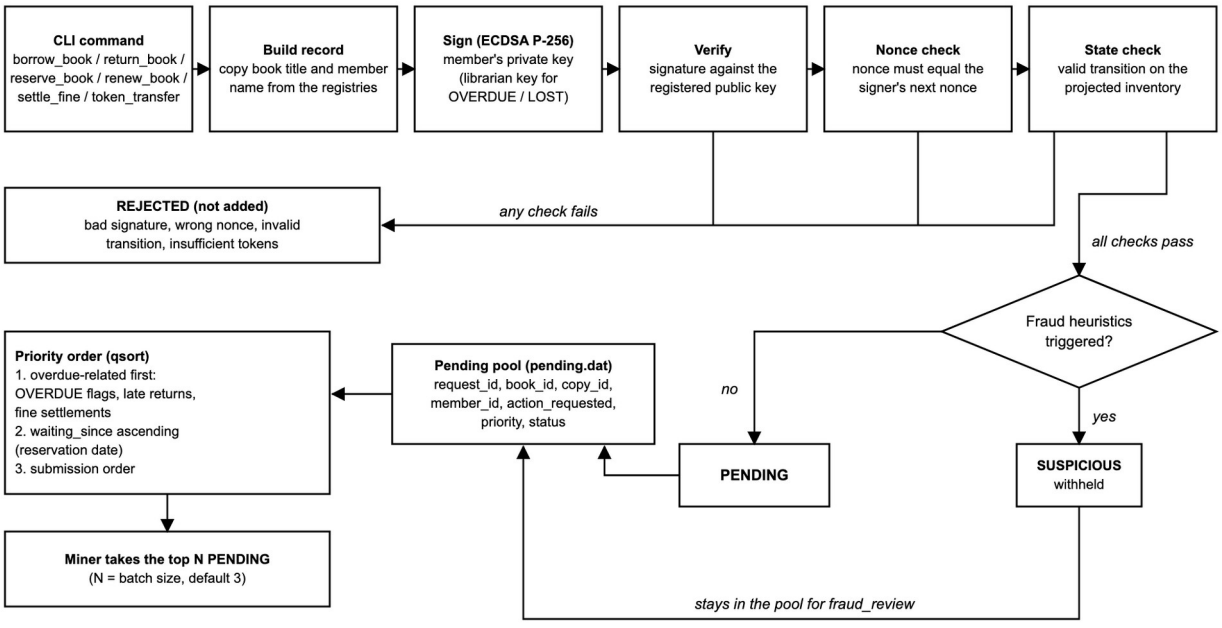
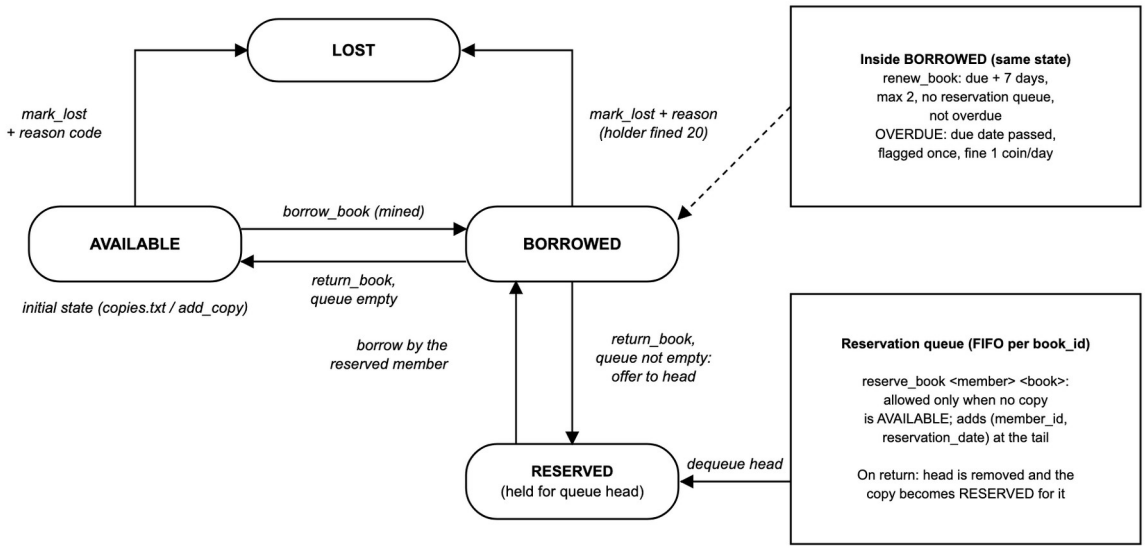


Figure 2. Pending pool flow with prioritisation and fraud flagging

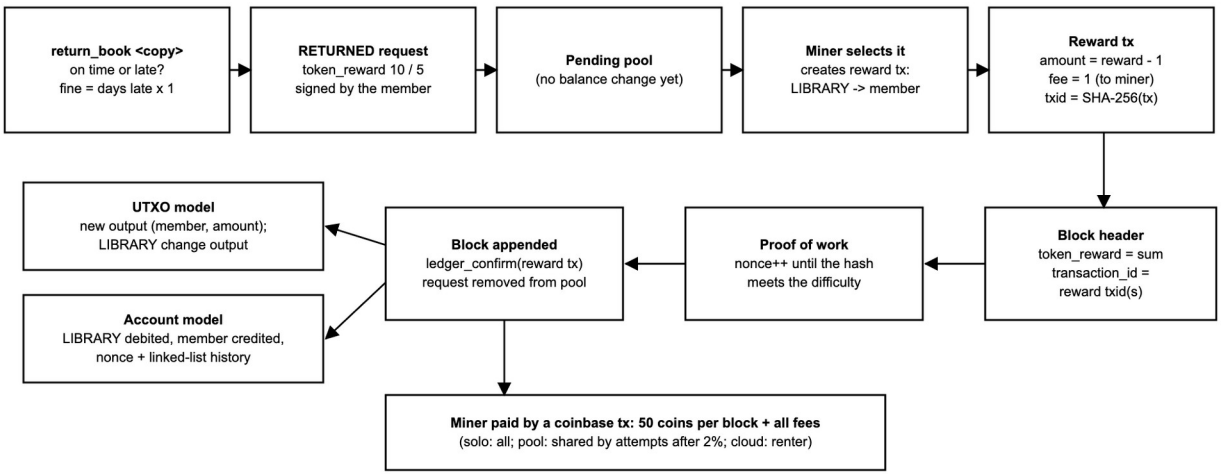
Figure 3. Copy inventory state machine and reservation queue



Only these transitions are accepted by state_apply(); anything else is rejected, both when a request is submitted and when the chain is replayed.

Figure 3. Copy inventory state machine and reservation queue

Figure 4. Token reward flow: from return to mining confirmation to balance update



A book that stays overdue and is not returned creates no reward transaction (OVERDUE record, token_reward 0).

Figure 4. Token reward flow from return to mining confirmation to balance update

Figure 6. Fraud detection pipeline: from submission to review

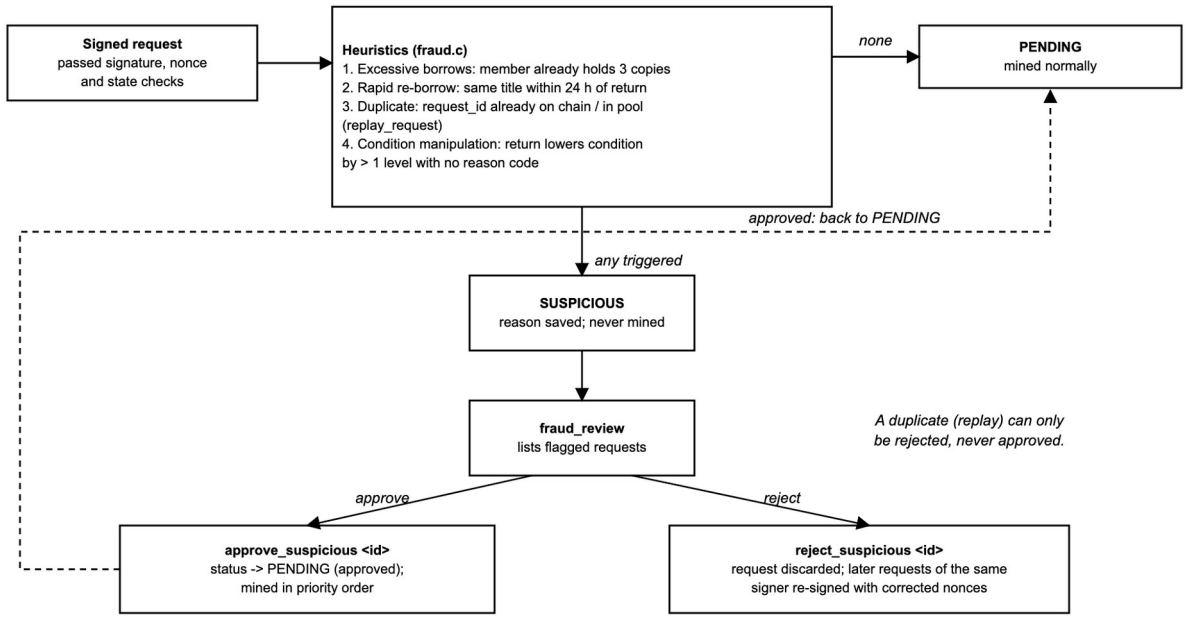


Figure 6. Fraud detection pipeline from submission to review

Figure 5 (the mining workflow) is shown in Section 11.

18. Testing Results

I tested the program in two ways. First, tests/run_tests.sh (make test) runs the real program in a temporary folder, types commands into it and checks the output. It has 76 checks in 10 groups: first run and registries, restart and reservations, overdue and fines, replay protection and fraud, the review workflow, both token models, pool and cloud mining and difficulty, registration, tamper detection, and access control. The result is:

```
$ make test
...
Result: 76 passed, 0 failed
```

Second, I ran the full demo flow by hand (the same steps as the demo video). Table 11 lists the scenarios with the expected and the actual output. All of the actual outputs are copied from that run.

Table 11: Test scenarios, expected vs actual

#	Scenario and steps	Expected	Actual output	Result
1	Invalid IDs: borrow_book ALU999 BK001, then borrow_book ALU001 BK999	Both rejected with a clear error; nothing added to the pool	ERROR: Member ALU999 not found in the member registry / ERROR: Book BK999 not found in the book registry	Pass
2	Batching and Merkle tree: 2 borrows + 1 reservation, then mine_solo and chain_view 1	One block with 3 records, attempts shown, Merkle root printed	Block body: 3 record(s) ... Proof of work found: nonce 159 after 160 hash attempts; Level 2 (root): 7ffb5b4540c4	Pass
3	Reservation and automatic	Return gives 10 coins;	reward tx e39d80a1: 10 coins (9.00 to member +	Pass

	offer: BK003 (1 copy) borrowed by ALU002, reserved by ALU003; ALU002 returns it	copy held for ALU003; others cannot borrow it	1.00 fee); RESERVATION OFFER: returned copy BK003-C1 is now RESERVED for ALU003	
4	Overdue, late return, fine: advance_days 20, return_book BK004-C1, settle_fine	Overdue flagged and mined first; late return gives 5 coins and a 6-coin fine; fine settled	OVERDUE DETECTED: copy BK004-C1 ... 6 day(s) past its due date; LATE by 6 day(s), fine 6 coin(s); 5 coins (4.00 to member + 1.00 fee); Owed 0, Paid 6	Pass
5	Replay protection: borrow_book ALU006 BK006 nonce=4; replay_request REQ000001	Wrong nonce rejected; replay flagged as duplicate and never mined	REJECTED: nonce 4 is not ALU006's expected next nonce 0; FLAGGED: duplicate request: request_id REQ000001 already exists on the chain	Pass
6	Fraud: ALU005 borrows a 4th copy; ALU001 returns NEW copy as WORN	Both SUSPICIOUS and withheld; approve releases one	SUSPICIOUS: excessive concurrent borrows: ALU005 already holds 3 copies; SUSPICIOUS: condition manipulation: BK001-C1 downgraded NEW -> WORN; Withheld REQ000018 (RETURNED): SUSPICIOUS	Pass
7	UTXO double spend: replay_tx e39d80a1 (a confirmed reward)	Rejected as a double spend	REJECTED: double spend rejected: input 45ef0a5e:0 is already spent	Pass
8	Difficulty 4: set_difficulty 4, then mine a transfer	Hash starts with 0000, about 65,536 attempts	Proof of work found: nonce 61535 after 61536 hash attempts (18.25 ms); Block hash: 000041357f08...	Pass
9	Tamper: tamper_block 2 record, then validate_chain, then a new borrow, then chain_load	Merkle and signature failures reported; session locked; reload restores a valid chain	Block #2: Merkle root recomputed ... does not match; record REQ000004: ECDSA signature ... does not verify; ERROR: Session locked; after chain_load: Chain is VALID	Pass

Scenario 4 output (overdue detection, late return and fine)

```
> advance_days 20
Library clock advanced by 20 day(s): it is now 2026-11-06 20:11 (simulated; ...).
OVERDUE DETECTED: copy BK004-C1 ('Half of a Yellow Sun') held by Didier (ALU004) is 6 day(s)
  past its due date; fine accrued so far: 6 coin(s).
SUBMITTED: request REQ000009 (OVERDUE) signed by LIB:librarian with nonce 1, ...
  [priority: overdue-related]
> return_book BK004-C1
Return: copy BK004-C1 ('Half of a Yellow Sun') by Didier (ALU004), due 2026-10-31: LATE by 6 day(s),
  fine 6 coin(s) at 1/day. Condition NEW -> NEW.
  Token reward when mined: 5 coins (late return) -> 4.00 to ALU004 after the 1.00-coin fee.
```

Scenario 6 output (fraud detection)

```
> borrow_book ALU005 BK005
FLAGGED: request REQ000017 (BORROWED) signed by ALU005 with nonce 3, ECDSA signature verified, ...
  SUSPICIOUS: excessive concurrent borrows: ALU005 already holds 3 copies (maximum 3)
> return_book BK001-C1 WORN
FLAGGED: request REQ000018 (RETURNED) signed by ALU001 with nonce 2, ...
  SUSPICIOUS: condition manipulation: BK001-C1 downgraded NEW -> WORN (2 levels) without a librarian
  override reason code
```

Scenario 9 output (tamper detection)

```
> tamper_block 2 record
Block #2 record REQ000004: due_date silently moved 30 days later (nothing recomputed).
> validate_chain
  Block #2: Merkle root recomputed from its 2 record(s) does not match merkle_root
  (a record was changed)
  Block #2: record REQ000004 (RENEWED): ECDSA signature by ALU001 does not verify
validate_chain: Chain is INVALID: 2 problem(s) found.
> borrow_book ALU002 BK002
ERROR: Session locked: the chain loaded from disk failed validate_chain. ...
```

Bugs found during testing: (1) fine payment records have no book, and validation wrongly reported them as unregistered; (2) a copy added with add_copy was treated as if it had existed from the start when the chain was replayed, so an older reservation looked invalid. I fixed this by storing the block index in copies.txt. Both now have regression checks in make test.

19. Limitations and Conclusion

- Member private keys are stored unencrypted (mode 0600) on the library machine, so the program can sign for members in this simulation. In a real system each member would keep their key on their own device.
- The program runs on one computer. There is no network and no consensus beyond proof of work, as the brief allows.
- Cloud rental fees are paid outside the chain, and the simulated clock (advance_days) exists only for demonstrations.
- The binary files depend on the compiler's struct layout (Section 15).

The final system meets the summative scope. Every lending action is a signed record. Each member signs with their own ECDSA key, and a nonce stops replays. Records wait in a prioritised pool, suspicious ones are held for review, and the rest are confirmed by proof of work in blocks with a Merkle root. Returns earn token rewards under either the UTXO or the account model. The copy inventory, reservations, renewals and fines follow a strict state machine. Everything is saved to disk and checked again when it is loaded, and any change to a record, a block, a key or a file is detected.